

PRACTICAL 2

AIM

Unguided Search Techniques: Write a python code for implementing Water Jug Problem using BFS.

THEORY

The Water Jug Problem is a classical problem in Artificial Intelligence that demonstrates the concept of state space search. The problem involves two water jugs with fixed capacities and an unlimited water supply. The objective is to measure a specific quantity of water using the allowed operations.

The possible operations include:

1. Filling a jug completely.
2. Emptying a jug completely.
3. Pouring water from one jug to another until one jug becomes empty or the other becomes full.

To solve this problem efficiently, Breadth First Search (BFS) can be used. BFS is an uninformed (unguided) search algorithm that explores the state space level by level. It starts from the initial state and explores all possible states that can be reached with one operation before moving to the next level.

Each state in the water jug problem can be represented as a pair (x, y) where:

- x represents the amount of water in Jug 1
- y represents the amount of water in Jug 2

BFS uses a queue data structure to keep track of states that need to be explored and a visited set to avoid revisiting the same states. The algorithm continues until the target amount of water is found in either of the jugs.

This approach guarantees finding the shortest sequence of operations required to reach the desired solution.

SOFTWARE USED

Jupyter Notebook

CODE

```
from collections import deque

def water_jug_bfs(jug1, jug2, target):
    visited = set()
    queue = deque([(0, 0)])

    while queue:
        x, y = queue.popleft()

        if (x, y) in visited:
            continue

        visited.add((x, y))
        print(x, y)

        if x == target or y == target:
            print("Target reached")
            return

    possible_states = [
        (jug1, y),
        (x, jug2),
        (0, y),
        (x, 0),
        (x - min(x, jug2 - y), y + min(x, jug2 - y)),
        (x + min(y, jug1 - x), y - min(y, jug1 - x))
    ]
```

```
        for state in possible_states:
            if state not in visited:
                queue.append(state)

    print("No solution found")

jug1 = 4
jug2 = 3
target = 2

water_jug_bfs(jug1, jug2, target)
```

OUTPUT

0 0

4 0

0 3

4 3

1 3

3 0

1 0

3 3

0 1

4 2

Target reached